ASTRA polytechnic
member of ASTRA

Kampus Merdeka INDONESIA JAYA
satu Indonesia
Semangat Astra Terpadu Untuk Indonesia

Data Type, Name, and Value

Prepared by : Radix R

Contents

- Basic/Primitive Type
- Composite Type
- Name
- Value
- Expression

```
PROGRAM Euclidean

{Given two non-negative integers m and n (m ≥ n). The Euclidean algorithm
looks for the greatest common divisor, GCD, of the two numbers, the largest
number that divides both of them without leaving a remainder.}

DECLARATION:
      m, n  : integer   { integers to be searched for their GCD }
      r     : integer   { remainder from division }

ALGORITHM:

Read (m, n)          { assume m ≥ n }
While n ≠ 0  do
      r  ← m mod n     { divide m with n, store the remainder in r }
      m ← n            { exchange m with n }
      n ← r            { exchange n with r }
End while
{ final condition after looping: n = 0 }

Write (m)
```

Data Type

- Programs generally manipulate objects / data in memory.

- Example of data type:
  - Numerical value
  - Character
  - String
  - Record

Data Type (cont.)

- Data types are are divided into 2 categories:
  1. Basic/Primitive type
  2. Composite type

- Data types associated with:
  1. Name
  2. Domain
  3. Operation & Operators

Basic/Primitive Type

1. Boolean
2. Integers
3. Floating point numbers / Real
4. Character

Boolean

1. Name : boolean
2. Domain : *true* or *false* ; {1,0}
3. Operation and Operators :

a	not a
true	false
false	true

a	b	a and b	a or b	a xor b
true	true	true	true	false
true	false	false	true	true
false	true	false	true	true
false	false	false	false	false

integer

1. Name : integer
2. Domain : -xyz, …, 0, …, xyz
3. Operation and Operators :

Arithmatic Operation:

+	Addition
–	Substraction
*	Multiplication
div	Division
mod	Modulo

Comparison Operation:

<	Less than
≤	Less than or equal to
>	Greater than
≥	Greater than or equal to
=	Equal to
≠	Not equal to

Floating point (*Real*)

1. Name : real / floating-point
2. Domain : 1.0×10^{-xy}, …, 1.0×10^{xy}
3. Operation and Operators :

Arithmatic Operation:

+	Addition
–	Substraction
*	Multiplication
/	Division

Comparison Operation:

<	Less than
≤	Less than or equal to
>	Greater than
≥	Greater than or equal to
≠	Not equal to

Character (*char*)

1. Name : char
2. Domain : every character on *keyboard*
3. Operation and Operators :

Comparison Operation:

<	Less than
≤	Less than or equal to
>	Greater than
≥	Greater than or equal to
=	Equal to
≠	Not equal to

Composite Type

1. String
2. A basic type that is given a new name
3. Structured type

String

1. Name : string
2. Domain : everything that defined in char
3. Operation and Operators :

Concatenation Operation:

operator : **+**

Example:

'Manajemen' + 'Informatika' = 'Manajemen Informatika'

Comparison Operation:

<	Less than
≤	Less than or equal to
>	Greater than
≥	Greater than or equal to
=	Equal to
≠	Not equal to

Basic Type with A New Name

1. purpose : sound ‘familiar’
2. Using the keyword : `type`
3. Example :

```
DECLARATION:
        type WholeNumber : integer

        M, N  : WholeNumber
        A     : real
```

Structured Type

1. The type in the form of *record*
2. Record consists of one or more *field*

field 1	*field* 2	*field* 3	...	*field* N

Example :

1. Defining date type:

dd	mm	yy

How to write date type:

```
type date : record
    < dd : integer,          { 1..31 }
      mm : integer,          { 1..12 }
      yy : integer           { > 0 }
    >
```

If D is a variable with date as the data type, then to access it's field we use :

```
D.dd
D.mm
D.yy
```

Name

1. Every object/creature has a name
2. Name is given because it will be easy to:
   - identify
   - refer
   - Differentiate from other objects
3. Name should be **unique**

Naming Rules

1. Names should start with alphabet letters, not numbers or other characters.
2. The characters used to build a name can only be alphabetic, numeric, and "_" (underscores).
3. Must not contain arithmetic, relational operators, punctuation marks and other special characters.
4. Must not be spaced.
5. Name length is unlimited.

Example of wrong naming

```
6point

  exam grade

    PT-1

     Day!

        A 1
```

Example of correct Naming

```
point6

 exam_grade or examGrade

    PT_1 or PT1

          day

              A1
```

Object that should be given a name

- Variable

```
DECLARATION:
    X, test_result, sum : real
    N : integer
    c : char
```

- Constant

```
DECLARATION:
    const phi = 3.14
    const Nmaks = 100
    const password = 'xyz'
```

- Structured Type

```
DECLARATION:
    type date : record
        <   dd : integer,          { 1..31 }
            mm : integer,          { 1..12 }
            yy : integer           { > 0 }
        >

    D     : date
```

Object that should be given a name (cont.)

```
PROGRAM CircleArea

{Given a circle with radius r, the program calculate the area A of the
circle. }

DECLARATION:
      Const Pi = 3.14    { Pi }
      r     : real       { radius of the circle }
      A     : real       { Area of the circle, Pi*r² }

ALGORITHM:

Read (r)                 { r > 0 }
A ← Pi * r * r           { calculate the circle area }
Write (A)
```

Object that should be given a name (cont.)

- Function

```
DECLARATION:
    FUNCTION Maximum(input A, B : integer) → integer
    { return a bigger number between A and B }
```

- Procedure

```
DECLARATION:
    PROCEDURE Exchange(input/output A, B : integer)
    { exchange the value of A and B }
```

Value

1. Is the amount of the data type that has been defined (basic type / composite)
2. The values can be:
    I. The content stored by a variable / constant
    II. The result of a calculation
    III. The value sent by a function

Storing value to a Variable

1. Direct:

```
Example:
     A ← 10          {the value of A is equal to 10}

     A ← B           {the value of A is equal to the value of B}

     A ← (B + C)/2   {the value of A is the result of (B + C)/2}
```

Notes:

The data type between the left side of the arrow and the right side should be the SAME.

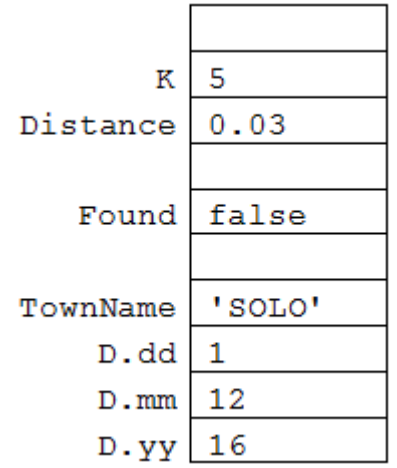
Storing value to a Variable (cont.)

1. Direct (cont.)

```
K ← 5
Distance ← 0.03
Found ← false
TownName ← 'SOLO'

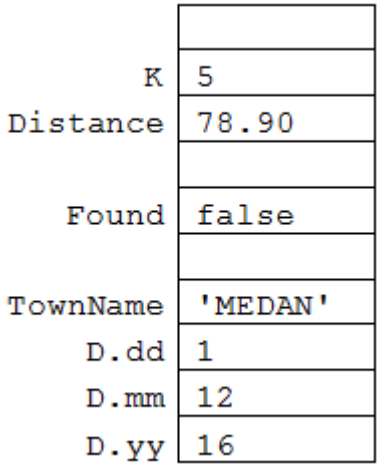
{ filling D date with 1/12/16 }
D.dd ← 1          { day 1 }
D.mm ← 12         { month 12 }
D.yy ← 16         { year 16 }
```

Memory state




```
Distance ← 78.90
TownName ← 'MEDAN'
```

Memory state



Storing value to a Variable (cont.)

Remember:

The value contained by a variable is its **LAST** value

Storing value to a Variable (cont.)

2. Reading

The variable value can be filled from Input devices (ie. keyboard).

Example:

```
read(student_name, ID, grade)
```

if the user typed:

```
Budi, 0320130001, 90.4
```

Memory state

student_name	'Budi'
ID	320130001
grade	90.4

Expression

Transformation of value into output is done through a computation.

Calculation is written in an expression.

The expression consists of **operands** and **operators**

There are 3 kinds of expressions:

1. Arithmetic expressions
2. Relational expression
3. String expression

Arithmatic Expression

Is an expression that the operand is numeric, and the result is numeric.

Ex.:

```
DECLARATION:
    a, b, c       : real
    d             : integer
    i, j, k       : integer

ALGORITHM:
    …

    c ← a * b                 {correct}

    d ← a * b                 {wrong}

    k ← ( i + j) div 2        {correct}
    ….
```

Relational / Boolean Expression

is the expression using this operators:

<, ≤, >, ≥, =, ≠, not, and, or, dan xor

The expression evaluation result is a value of boolean (true or false)

ex:

```
DECLARATION:
    exist, found, result    : boolean
    x, y                    : integer

ALGORITHM:

    found ← false
    exist ← true
    x ← 8
    y ← 12

    {example: relasional/boolean expression}

    result ← not exist                          {false}
    result ← exist or found                     {true}
    result ← exist and true                     {true}
    result ← x < 5                              {false}
    result ← exist or (x = y)                   {true}

    Write(result)
```

String Expression

It is an operand expression of type char / string, with operator "+" (concatenation).

Ex:

```
DEKLARASI
    Kar                         : char
    S                           : string


ALGORITMA
    {contoh ekspresi string}

    S <- 'jl. Mawar'
    Kar <- ' '                  {beri nilai spasi}
    S <- (S + Kar) + 'c'
```

Writing Value into Output Device

The values that exist in the memory can be displayed to the output device (eg screen).

Algoritma notation that is used : write

```
PROGRAM Euclidean

{Given two non-negative integers m and n  (m ≥ n).  The Euclidean algorithm
looks for the greatest common divisor, GCD, of the two numbers, the largest
number that divides both of them without leaving a remainder.}

DECLARATION:
      m, n   : integer    { integers to be searched for their GCD }
      r      : integer    { remainder from division }

ALGORITHM:

Read (m, n)          { assume m ≥ n }
While n ≠ 0  do
      r  ← m mod n      { divide m with n, store the remainder in r }
      m ← n             { exchange m with n }
      n ← r             { exchange n with r }
End while
{ final condition after looping: n = 0 }

Write (m)
```

Ex.

After running, what is the value of M and P?

```
DECLARATION:
    M, P    : integer

ALGORITHM:
    M ← 16
    P ← M * 2
    M ← P + 100
    P ← P + M
```

M = 132 and P = 164

Ex.

Write an algorithm for this problem

```
Algorithm calculate_the_average_value
(Calculates the average value of a set of numbers that are read from
an input device )
```

Ex.

Create a complete algorithm notation for this:

$$m = \frac{a-b}{3ac}\left(1-\frac{b}{cd}\right)$$

Ex.

After running, what is the value of m?

```
PROGRAM JustATest

{ just some test program }

DECLARATION:
      m      : integer
      P      : integer

ALGORITHM:

P  ← 10
P  ← P + 10
m  ← m + P

Write (m)
```

What is wrong?

Bibliography

- Munir, Lidya, “Algoritma & Pemrograman”, 6th Ed, 2016 Penerbit Informatika.
- Kernighan, Ritchie, “The C Programming Language”, 2nd Ed, Prentice Hall.